GUÍA DEL ESTUDIANTE

Práctica 3 — Reglas, árbol y SVM: precisión, recall y umbral

Duración	Práctica principal: 120 minutos. Extensión (botnet): 60 minutos. Cada parte opcional: unos 15 minutos
Modalidad	Individual
Archivos del dataset	`Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv` (principal) · `Friday-WorkingHours-Morning.pcap_ISCX.csv` (extensión) · `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv` (opcional)
Paquete	`laboratorio3_estudiante.zip` con los scripts, la plantilla y esta guía
Requisitos	Python 3.10 o superior y las librerías `pandas`, `numpy`, `matplotlib` y `scikit-learn`. Se recomiendan 8 GB de RAM
Entregable	Un .zip con el informe con las respuestas, las tablas completadas y los gráficos, más tu archivo `laboratorio3_plantilla.py` terminado

1. ¿De qué trata esta práctica?

Eres analista en un SOC y te piden detectar ataques web en tráfico de red (fuerza bruta, XSS e inyección SQL), que son una fracción mínima del tráfico: alrededor del 1,3 % de los flujos. En esta práctica reúnes todo lo visto hasta ahora en un solo ejercicio:

- Una regla escrita por una persona frente a un modelo, y la tasa base.
- Datos limpios, columnas problemáticas, partición correcta y cuidado con los atajos.
- Árbol de decisión, SVM, precisión, recall, F1 y, sobre todo, el umbral.

La idea central: un modelo no entrega un sí o un no, entrega una probabilidad. El umbral decide desde qué probabilidad se genera una alerta. Moverlo cambia cuántos ataques detectas y cuántas falsas alarmas provocas. Aprenderás a elegirlo con un criterio, a comprobarlo con datos nuevos y a descubrir que un modelo más complejo no siempre es mejor.

2. Objetivos de aprendizaje

Al terminar serás capaz de:

- Implementar precisión, recall y F1 a partir de TP, FP y FN, y verificarlos con scikit-learn.
- Evaluar una regla escrita a mano, un árbol de decisión (con sus reglas) y un SVM lineal sobre los mismos datos.
- Medir y graficar cómo cambian la precisión, el recall, los FP y los FN al mover el umbral.
- Elegir un umbral con un criterio explícito usando un conjunto de validación y evaluarlo una sola vez en el conjunto de prueba.
- Interpretar una curva precisión-recall y el AP (average precision).
- Explicar con datos por qué un modelo más complejo (SVM) no es necesariamente mejor que uno simple (árbol).

3. Conceptos que necesitas

Concepto	Qué significa	En esta práctica
TP, FP, FN, TN	Aciertos y errores del detector: ataque detectado, falsa alarma, ataque perdido, benigno ignorado	La clase positiva es ataque (1)
Precisión	De lo que marcaste como ataque, qué fracción lo era: `TP / (TP + FP)`	Qué tan confiable es una alerta
Recall	De los ataques que existen, qué fracción detectaste: `TP / (TP + FN)`	Cuántos ataques no se escapan
F1	Media armónica de precisión y recall: `2·P·R / (P + R)`	Resumen de ambos en un número
Umbral	Probabilidad mínima para generar una alerta: se alerta si `probabilidad ≥ umbral`	Lo moverás de 0,05 a 0,95
Regla	Condición escrita por una persona	`Destination Port == puerto`
Árbol de decisión	Secuencia de preguntas aprendida de los datos; cada camino hasta una hoja es una regla	Profundidad 8, mín. 20 por hoja
SVM lineal	Busca la frontera lineal que mejor separa las clases; se calibra para obtener probabilidades	Ya viene armado en `p3_utils.py`
Partición 60/20/20	Entrenamiento (ajusta el modelo), validación (elige el umbral), prueba (se mira una sola vez)	Estratificada: igual proporción de ataques
Curva PR y AP	Precisión frente a recall para todos los umbrales; el AP resume la curva en un valor (1,0 = perfecto)	Compararás modelos con ella

4. Archivos del dataset a trabajar

Se trabaja con la carpeta **MachineLearningCVE** del dataset CIC-IDS2017 (no con TrafficLabelling). Necesitas estos archivos, con **estos nombres exactos**:

Archivo	Clave	Qué contiene	Uso
`Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv`	`web`	Fuerza bruta web, XSS e inyección SQL (≈ 1,3 % de ataques)	**Principal**
`Friday-WorkingHours-Morning.pcap_ISCX.csv`	`bot`	Botnet (≈ 1,1 % de ataques)	**Extensión**
`Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv`	`ddos`	DDoS (≈ 57 % de ataques)	Opcional (contraste)

Los dos primeros son obligatorios. El tercero es el archivo del Laboratorio 2: si ya lo tienes, cópialo también.

5. Cómo preparar la colección

A. Descomprime el paquete y revisa la estructura

Descomprime `laboratorio3.zip` en una carpeta con tu nombre y apellido sin espacios en la ruta (por ejemplo `C:\juan_perez`). Debe quedar así:

```
laboratorio3/
  LEEME.txt
  guia_estudiante_practica_3.pdf
  preparar_datos.py          <- limpia los CSV (no lo modifiques)
  p3_utils.py                <- utilidades y gráficos (no lo modifiques)
  laboratorio3_plantilla.py     <- AQUÍ trabajas: completas los TODO
  requerimientos.txt
  datos_originales/          <- copia aquí los CSV
  datos_limpios/             <- se llena solo
  figuras/                   <- se llena solo
```

B. Copia los CSV a datos_originales

Copia los archivos de la sección 4 (carpeta MachineLearningCVE de tu descarga) dentro de `datos_originales/`. **No cambies los nombres.**

C. Instala las librerías

```
pip install -r requerimientos.txt
# o bien: pip install pandas numpy matplotlib scikit-learn
```

D. Ejecuta la limpieza

```
cd laboratorio3
python preparar_datos.py
```

El script aplica la limpieza de la práctica 2.2: quita espacios de los nombres, elimina infinitos, valores faltantes y duplicados, borra la columna repetida `Fwd Header Length.1` y las columnas constantes, y crea la variable `es_ataque`. Crea `web_limpio.csv.gz`, `bot_limpio.csv.gz` y, si existe, `ddos_limpio.csv.gz` en `datos_limpios/`. **Comprueba que tus cifras coinciden:**

Clave	Filas (antes → después)	Ataques	% de ataques	Características
`web`	170.366 → 164.179	2.143	1,305 %	67
`bot`	191.033 → 184.044	1.948	1,058 %	67
`ddos`	225.745 → 223.082	128.014	57,384 %	67

6. Contenido del paquete y cómo se trabaja

Archivo	Para qué sirve
`preparar_datos.py`	Limpia los CSV originales y genera los archivos de `datos_limpios/`. Se ejecuta una sola vez
`p3_utils.py`	Carga de datos, partición 60/20/20, el modelo SVM y todas las funciones de gráficos. No lo modifiques
`laboratorio3_plantilla.py`	Tu archivo de trabajo. Contiene 6 funciones con TODO para completar y el flujo de la práctica
`datos_limpios/` y `figuras/`	Carpetas de salida: datos listos para modelar; gráficos PNG y tablas CSV que genera el script

La plantilla se ejecuta de principio a fin con un solo comando. Los pasos que dependen de una función que aún no completaste se omiten y el script te lo avisa con `[PENDIENTE]`, de modo que puedes avanzar paso a paso:

```
python laboratorio3_plantilla.py --dataset web                # práctica principal
python laboratorio3_plantilla.py --dataset bot                # extensión
python laboratorio3_plantilla.py --dataset web --prevalencia  # parte opcional A
python laboratorio3_plantilla.py --dataset ddos               # parte opcional B
```

Los **6 TODO** que debes implementar: `metricas`, `evaluar_regla_puerto`, `entrenar_arbol`, `reglas_texto`, `barrido_umbral` y `elegir_umbral`. Cada función explica en su descripción qué recibe y qué devuelve. Todos los gráficos se generan solos: los encuentras en `figuras/`.

Gráfico (carpeta figuras/)	Qué muestra
`<clave>_01_clases.png`	Cantidad de benignos y ataques en cada conjunto
`<clave>_02_arbol_reglas.png`	El árbol de profundidad 3: cada camino es una regla
`<clave>_03_barrido_svm.png` y `_arbol.png`	Precisión, recall y F1 según el umbral, con los falsos positivos en barras y el umbral elegido en verde
`<clave>_04_curvas_pr.png`	Curvas precisión-recall en validación (con zoom a la zona 0,8 a 1,0) y la regla como punto
`<clave>_05_comparacion_umbral05.png`	Regla, árbol y SVM con umbral 0,5
`<clave>_06_matriz_*.png`	Matriz de confusión en prueba con el umbral elegido
`<clave>_07_curvas_pr_prueba.png`	Curvas precisión-recall en prueba
`<clave>_08_prevalencia.png`	Efecto de que los ataques sean más raros (opcional)

7. Desarrollo guiado

Anota tus respuestas con el número de la pregunta (P1, P2…) y completa las tablas de la sección 8 mientras avanzas.

Paso 0 · Preparación y primera ejecución

Abre una terminal en la carpeta `laboratorio3/` y ejecuta la plantilla sin modificar nada:

```
python laboratorio3_plantilla.py --dataset web
```

Verás varios mensajes `[PENDIENTE]`: es lo esperado. Abre `laboratorio3_plantilla.py` y revisa las seis funciones con TODO. Abre también `figuras/web_01_clases.png`, el único gráfico que se genera todavía.

Paso 1 · Precisión y recall desde cero (TODO 1)

Implementa `métricas (y_real, y_pred)`: cuenta TP, FP, FN y TN y calcula precisión, recall y F1 con las fórmulas de la sección 3. Si un denominador es 0, la métrica vale 0,0. Al ejecutar, el Paso 0 del script compara tu función con scikit-learn y muestra ✓ o ✗. **No continúes hasta ver `✓`.**

Preguntas

- **P1.** En este problema, ¿qué significan TP, FP, FN y TN? Da un ejemplo de cada uno con un flujo de red.
- **P2.** ¿Qué error es más costoso en un SOC, un FP o un FN? ¿De qué depende?

Paso 2 · Datos y partición

Ejecuta de nuevo. Comprueba que la partición coincide con estos valores y observa `web_01_clases.png`:

Conjunto	Flujos	Ataques
Entrenamiento	98.507	1.286
Validación	32.836	429
Prueba	32.836	428

Preguntas

- **P3.** ¿Qué porcentaje de los flujos son ataques y cuántos ataques hay en validación y en prueba? ¿Es una proporción realista?
- **P4.** ¿Por qué usamos tres conjuntos (entrenamiento, validación y prueba) y no solo dos? ¿Por qué la partición es estratificada?

Paso 3 · Una regla escrita a mano (TODO 2)

Implementa `evaluar_regla_puerto(X, y, puerto)`: la regla predice ataque cuando `Destination Port == puerto` y devuelve el resultado de tu `metricas()`. El script elige solo el puerto más frecuente entre los ataques del entrenamiento y evalúa la regla en validación.

Preguntas

- **P5.** ¿Qué precisión y recall obtiene la regla del puerto? ¿Qué te dice cada valor?
- **P6.** ¿Pondrías esta regla en producción? ¿Qué le agregarías?

Paso 4 · El árbol de decisión y sus reglas (TODO 3 y 4)

Implementa `entrenar_arbol` (árbol con `max_depth=8`, `min_samples_leaf=20`, `random_state=0`) y `reglas_texto` (árbol de profundidad 3 más `export_text`). Ejecuta, lee las reglas que imprime (`class: 1` = ataque) y abre `web_02_arbol_reglas.png`. El parámetro `min_samples_leaf` obliga a que cada hoja tenga varias muestras; fíjate después qué efecto tiene sobre las probabilidades.

Preguntas

- **P7.** Escribe con tus palabras una de las reglas del árbol de profundidad 3. ¿Qué variables usa?
- **P8.** ¿Cuántas probabilidades distintas produce el árbol? ¿Qué implica para el umbral?
- **P9.** ¿Qué ventaja tiene el árbol frente al SVM en cuanto a explicación? ¿Y qué desventaja?

Paso 5 · El SVM lineal

El SVM ya está armado en la función `modelo_svm()` de `p3_utils.py`: léela. No tienes que implementar nada; ejecuta y analiza lo que imprime el script.

Preguntas

- **P10.** ¿Cuántas probabilidades distintas produce el SVM? ¿Por qué es relevante?
- **P11.** El modelo SVM aplica log(1+x), estandarización y calibración. ¿Para qué sirve cada paso?

Paso 6 · Mover el umbral (TODO 5)

Implementa `barrido_umbral(y, scores, umbrales)`: para cada umbral predice ataque cuando `score >= umbral`, usa tu `metricas()` y devuelve una tabla con `umbral, precision, recall, f1, fp, fn, alertas` (alertas = TP + FP). Ejecuta y abre `web_03_barrido_svm.png`, `web_03_barrido_arbol.png` y `web_04_curvas_pr.png` (mira el zoom de la derecha). El script comprueba que el recall no aumenta al subir el umbral. Completa la Tabla A.

Preguntas

- **P12.** Al subir el umbral del SVM de 0,1 a 0,9, ¿qué pasa con la precisión, el recall, los FP y los FN? Explica por qué.
- **P13.** Completa la tabla de umbrales (0,1; 0,3; 0,5; 0,7; 0,9) para el SVM y el árbol. ¿Dónde está el «codo» del SVM?
- **P14.** ¿Por qué en el árbol varios umbrales dan exactamente la misma fila? ¿Qué umbrales no vale la pena probar por separado?
- **P15.** En las curvas precisión-recall (con zoom), ¿qué modelo se ve mejor en validación? ¿Qué significa AP (average precision)?

Paso 7 · Elegir el umbral (TODO 6)

Implementa `elegir_umbral(y, scores, criterio, recall_min)` con dos criterios: `recall_min` (entre los umbrales con recall ≥ 0,95, el de mayor precisión) y `f1` (el de mayor F1). Si hay empate, elige el umbral más alto. Si ningún umbral llega al recall pedido, devuelve el de mayor recall e indica `False`. Completa la Tabla B.

Preguntas

- **P16.** ¿Por qué el umbral se elige en validación y no en prueba?
- **P17.** Completa: umbral elegido y métricas en validación para cada modelo y criterio. ¿Qué diferencia hay entre «recall ≥ 0,95» y «máximo F1»?
- **P18.** Elige el criterio para (a) un SOC pequeño con un solo analista y (b) una empresa con activos críticos. Justifica con números.

Paso 8 · Evaluación final en el conjunto de prueba

El script evalúa una sola vez en prueba cada detector con los umbrales elegidos y compara con el umbral 0,5. Observa la tabla final, las matrices de confusión (`web_06_matriz_*.png`) y `web_07_curvas_pr_prueba.png`. Completa la Tabla C.

Preguntas

- **P19.** ¿Los resultados en prueba son coherentes con los de validación? ¿Cuál es la mayor diferencia?
- **P20.** En el caso web, ¿qué detector desplegarías y por qué? ¿Es el SVM mejor que el árbol aquí?
- **P21.** Compara la regla del puerto con los modelos: ¿qué ganó y qué perdió la regla?

Paso 9 · Extensión: botnet, ¿es el SVM siempre mejor?

No hay nada nuevo que programar: ejecuta el mismo código con otro archivo y compara:

```
python laboratorio3_plantilla.py --dataset bot
```

Revisa los gráficos con prefijo `bot_` (sobre todo `bot_03_barrido_svm.png` y `bot_07_curvas_pr_prueba.png`), y completa la Tabla D. Fíjate en si el script advierte que algún criterio no se cumple.

Preguntas

- **P22.** Ejecuta la práctica con el archivo de botnet. ¿Qué precisión y recall tiene la regla del puerto 8080? ¿En qué se diferencia de la regla en el caso web?
- **P23.** ¿Qué ocurre con el SVM al subir o bajar el umbral? ¿Hay un tope de recall? Describe el codo.
- **P24.** Compara el árbol y el SVM en botnet (prueba). ¿Cuál es mejor y por qué?
- **P25.** ¿Qué concluyes sobre la idea de que «el SVM es mejor porque es más completo o más complejo»? Si alguien propone bajar el umbral del SVM a 0,04 para detectar el 95 % de los bots, ¿qué responderías?

Paso 10 · Opcional A: cuando los ataques son cada vez más raros

Ejecuta con la opción `--prevalencia` (en web y, si quieres, en botnet). El script simula tener menos ataques en la prueba, sin cambiar el umbral, y grafica el efecto (`web_08_prevalencia.png`). Completa la Tabla E.

Preguntas

- **P26.** Con el umbral fijo, ¿qué pasa con la precisión y con el recall a medida que los ataques son más raros? ¿Por qué el árbol resiste mejor?

Paso 11 · Opcional B: contraste con DDoS

Ejecuta `python laboratorio3_plantilla.py --dataset ddos` y observa las tablas de barrido y las curvas. Recuerda lo que viste en la práctica 2.2 sobre este archivo.

Preguntas

- **P27.** Ejecuta la práctica con el archivo DDoS. ¿Qué pasa con precisión y recall al mover el umbral? ¿Qué te dice del dataset?

Paso 12 · Reflexión final

Preguntas

- **P28.** Escribe una reflexión de 8 a 10 líneas con tus propias palabras: ¿qué umbral y qué criterio usarías?, ¿qué aprendiste sobre complejidad del modelo y desempeño?, ¿qué limitaciones tiene este dataset?

8. Tablas para completar

Tabla A · Efecto del umbral en validación (web)

SVM lineal

Umbral	Precisión	Recall	FP	FN	Alertas
0,1					
0,3					
0,5					
0,7					
0,9					

Árbol de decisión

Umbral	Precisión	Recall	FP	FN	Alertas
0,1					
0,3					
0,5					
0,7					
0,9					

Tabla B · Umbral elegido en validación (web)

Modelo y criterio	Umbral	Precisión	Recall	FP	FN
SVM · recall ≥ 0,95					
SVM · máximo F1					
Árbol · recall ≥ 0,95					
Árbol · máximo F1					

Tabla C · Resultado final en prueba (web)

Detector	Umbral	Precisión	Recall	FP	FN
Regla de puerto	—				
Árbol (umbral 0,5)	0,5				
Árbol (umbral elegido)					
SVM (umbral 0,5)	0,5				
SVM (umbral elegido)					

Tabla D · Resultado final en prueba (botnet)

Detector	Umbral	Precisión	Recall	FP	FN
Regla de puerto	—				
Árbol (umbral 0,5)	0,5				
Árbol (umbral elegido)					
SVM (umbral 0,5)	0,5				
SVM (umbral elegido)					

Tabla E (opcional) · Efecto de la proporción de ataques

Ataques en el tráfico	Precisión SVM	Precisión árbol	FP SVM	FP árbol
1,3 %				
0,5 %				
0,2 %				
0,1 %				

9. Qué debes entregar

Entregas **un solo archivo .zip** llamado `apellido_nombre_laboratorio3.zip` que contenga tu carpeta `laboratorio3/` con:

```
laboratorio3/
  laboratorio3_plantilla.py     <- terminado: los 6 TODO resueltos
  p3_utils.py                <- sin modificar
  informe_laboratorio3.docx     <- tu informe en Word
  figuras/                   <- PNG y CSV generados por TU script, de web y de bot
```

- Informe (`informe_laboratorio3.docx`): respuestas P1 a P25 y P28 escritas con el formato «P1. …», «P2. …» (P26 y P27 son opcionales), y las tablas A, B, C y D copiadas de la sección 8 y completadas (la E es opcional) **EN ESE ORDEN**.
- Al menos cinco gráficos de `figuras/` dentro del informe, cada uno con un comentario de dos o tres líneas: el barrido del SVM (web), las curvas PR (web), la matriz de confusión elegida (web), el barrido del SVM (botnet) y las curvas PR en prueba (botnet).
- Plantilla: `laboratorio3_plantilla.py` con los 6 TODO completos, que se ejecute de principio a fin con `--dataset web` y con `--dataset bot`.
- Figuras: ejecuta tu plantilla para web y para bot antes de comprimir, para que `figuras/` tenga todos los archivos (tablas CSV y gráficos de las dos pruebas).

Cómo armar el .zip

- Dentro de `laboratorio3/`, vacía las carpetas `datos_originales/` y `datos_limpios/` (son archivos muy grandes y no se entregan).
- Comprueba que quedan la plantilla, `p3_utils.py`, el informe `.docx` y la carpeta `figuras/`.
- Comprime la carpeta `laboratorio3/` completa. En Windows: clic derecho sobre la carpeta → Enviar a → Carpeta comprimida. Renombra el zip como `apellido_nombre_laboratorio3.zip`.
- Entrega el zip por el medio indicado por el docente.
- **No cambies los nombres de los archivos ni de las funciones**

Criterios de evaluación

Criterio	Qué se valora	Peso
Implementación	Las seis funciones están correctas y el script corre sin errores (el Paso 0 muestra ✓)	25 %
Análisis del umbral	Tablas A y B correctas; explica el compromiso precisión-recall con números (P12 a P18)	25 %
Evaluación final y decisión	Tabla C, coherencia entre validación y prueba, decisión justificada (P19 a P21)	15 %
Extensión botnet	Tabla D y conclusión argumentada sobre complejidad frente a desempeño (P22 a P25)	20 %
Reflexión	Visión crítica del método y de las limitaciones del dataset (P28)	15 %

10. Problemas frecuentes

Síntoma	Posible causa	Qué hacer
`FileNotFoundError: datos_limpios/web_limpio.csv.gz`	No ejecutaste la limpieza	Ejecuta `python preparar_datos.py` y revisa que no falten archivos
`preparar_datos.py` dice «NO ENCONTRADO»	El nombre del CSV no es exacto o no está en la carpeta	Copia los archivos con los nombres de la sección 4 a `datos_originales/` (carpeta MachineLearningCVE)
Error con una ruta cortada en un espacio	La ruta tiene espacios sin comillas	Escribe la ruta entre comillas dobles
`ModuleNotFoundError`	Falta una librería	`pip install -r requirements.txt`
`[PENDIENTE] TODO n`	Aún no implementas esa función	Es lo esperado: completa el TODO indicado
✗ `metricas() NO coincide`	Error en TP, FP, FN o en una división por cero	Revisa las fórmulas y el caso en que no hay predicciones positivas
Mis números no coinciden con los de la guía	Cambiaste un parámetro del árbol o usaste otro CSV	Usa `max_depth=8`, `min_samples_leaf=20`, `random_state=0` y los CSV originales; no cambies la partición
El SVM tarda un poco	Se calibra con validación cruzada sobre ≈ 100 mil flujos	Es normal: de 10 a 40 segundos
No veo los gráficos	Se guardan como imágenes	Ábrelos desde la carpeta `figuras/`

11. Para ir más allá (opcional)

- Cambia `profundidad` y `min_hoja` del árbol (por ejemplo, 3, 5, 12 y 5, 50, 200) y observa cómo cambian el número de probabilidades distintas y la curva PR.
- Prueba `class_weight="balanced"` en el árbol y compara con el umbral 0,5.
- Calcula el recall por tipo de ataque web (fuerza bruta, XSS, inyección SQL) con la columna `Label` del archivo limpio. ¿Cuál es el más difícil de detectar? ¿Por qué la inyección SQL no sirve para evaluar?
- Usa la carpeta TrafficLabelling, que incluye marcas de tiempo, para hacer una partición temporal (entrenar con lo anterior y probar con lo posterior) y compara con la aleatoria.
- Prueba un SVM con kernel no lineal sobre una muestra de 20.000 flujos y compara tiempo y rendimiento con el SVM lineal y el árbol.